

DIAGRAMAS UML E ARQUITETURA

Todos os diagramas em código fonte para renderização

INSTRUÇÕES DE USO

PlantUML

Renderize online em: <https://www.plantuml.com/plantuml/uml/> Ou use extensão VS Code: "PlantUML" by jebbs

Mermaid

Renderize online em: <https://mermaid.live/> Ou visualize diretamente no GitHub/VS Code

DIAGRAMA 1: SIMPLE FACTORY - CLASS DIAGRAM

plantuml

```

@startuml SimpleFactory
!theme plain

interface INotifier {
    +Send(recipient: string, message: string): void
}

class EmailNotifier {
    +Send(recipient: string, message: string): void
}

class SmsNotifier {
    +Send(recipient: string, message: string): void
}

class NotifierFactory {
    +{static} Create(type: string): INotifier
}

INotifier <|.. EmailNotifier
INotifier <|.. SmsNotifier
NotifierFactory ..> INotifier : creates
NotifierFactory ..> EmailNotifier : creates
NotifierFactory ..> SmsNotifier : creates

note right of NotifierFactory
    Simple Factory
    - Método estático
    - Acoplamento forte
    - Sem extensibilidade
end note

@enduml

```

DIAGRAMA 2: FACTORY METHOD - CLASS DIAGRAM

```

plantuml

```

@startuml FactoryMethod

!theme plain

interface INotifier {

 +Send(recipient: string, message: string): void

}

abstract class NotificationService {

 #{abstract} CreateNotifier(): INotifier

 +NotifyCustomer(recipient: string, message: string): void

}

class EmailNotificationService {

 #CreateNotifier(): INotifier

}

class SmsNotificationService {

 #CreateNotifier(): INotifier

}

class EmailNotifier {

 +Send(recipient: string, message: string): void

}

class SmsNotifier {

 +Send(recipient: string, message: string): void

}

INotifier <|.. EmailNotifier

INotifier <|.. SmsNotifier

NotificationService <|-- EmailNotificationService

NotificationService <|-- SmsNotificationService

NotificationService ..> INotifier : uses

EmailNotificationService ..> EmailNotifier : creates

SmsNotificationService ..> SmsNotifier : creates

note right of NotificationService

 Factory Method Pattern

- Template Method usa Factory Method

- Subclasses decidem qual produto criar

- Extensível via herança

end note

@enduml

DIAGRAMA 3: ABSTRACT FACTORY - CLASS DIAGRAM

plantuml

@startuml AbstractFactory

!theme plain

' Produtos abstratos

interface IReportHeader {

 +Render(title: string): void

}

interface IReportTable {

 +Render(data: string[][]): void

}

interface IReportChart {

 +Render(values: double[]): void

}

' Abstract Factory

interface IReportFactory {

 +CreateHeader(): IReportHeader

 +CreateTable(): IReportTable

 +CreateChart(): IReportChart

}

' Concrete Factories

class PdfReportFactory {

 +CreateHeader(): IReportHeader

 +CreateTable(): IReportTable

 +CreateChart(): IReportChart

}

class ExcelReportFactory {

 +CreateHeader(): IReportHeader

 +CreateTable(): IReportTable

 +CreateChart(): IReportChart

}

' Produtos concretos - Família PDF

class PdfReportHeader

class PdfReportTable

class PdfReportChart

' Produtos concretos - Família Excel

class ExcelReportHeader

class ExcelReportTable

class ExcelReportChart

' Cliente

class ReportGenerator {

```

-factory: IReportFactory
+GenerateReport(title, data, chartData): void
}

IReportFactory <|.. PdfReportFactory
IReportFactory <|.. ExcelReportFactory

IReportHeader <|.. PdfReportHeader
IReportTable <|.. PdfReportTable
IReportChart <|.. PdfReportChart

IReportHeader <|.. ExcelReportHeader
IReportTable <|.. ExcelReportTable
IReportChart <|.. ExcelReportChart

PdfReportFactory ..> PdfReportHeader : creates
PdfReportFactory ..> PdfReportTable : creates
PdfReportFactory ..> PdfReportChart : creates

ExcelReportFactory ..> ExcelReportHeader : creates
ExcelReportFactory ..> ExcelReportTable : creates
ExcelReportFactory ..> ExcelReportChart : creates

ReportGenerator --> IReportFactory : uses

note bottom of IReportFactory
    Abstract Factory
    - Cria famílias de produtos relacionados
    - Garante compatibilidade entre produtos
    - Facilita troca de família inteira
end note

@enduml

```

DIAGRAMA 4: INTEGRAÇÃO COM DI - CLASS DIAGRAM

```

plantuml

```

@startuml DIIntegration

!theme plain

```
interface IPaymentProcessor {  
    +Process(payment: Payment): PaymentResult  
}
```

```
class CreditCardProcessor {  
    -logger: ILogger  
    +Process(payment: Payment): PaymentResult  
}
```

```
class BoletoProcessor {  
    -logger: ILogger  
    +Process(payment: Payment): PaymentResult  
}
```

```
interface IPaymentFactory {  
    +Create(type: string): IPaymentProcessor  
}
```

```
class PaymentFactory {  
    -serviceProvider: IServiceProvider  
    +Create(type: string): IPaymentProcessor  
}
```

```
class PaymentService {  
    -factory: IPaymentFactory  
    +ProcessPayment(payment: Payment): PaymentResult  
}
```

```
class IServiceProvider <<framework>> {  
    +GetRequiredService<T>(): T  
}
```

IPaymentProcessor <|.. CreditCardProcessor

IPaymentProcessor <|.. BoletoProcessor

IPaymentFactory <|.. PaymentFactory

PaymentFactory --> IServiceProvider : uses

PaymentService --> IPaymentFactory : uses

PaymentFactory ..> IPaymentProcessor : creates

note right of PaymentFactory

Factory com DI

- Usa IServiceProvider para resolver

- Não instancia diretamente

- Respeita lifetimes do container

end note

@enduml

DIAGRAMA 5: PROJECT FACTORY - CLASS DIAGRAM

plantuml

@startuml ProjectFactory

!theme plain

```
interface IModule {
    +Name: string
    +Version: string
    +ConfigureServices(services: IServiceCollection): void
    +Initialize(provider: IServiceProvider): void
}
```

```
interface IProjectFactory {
    +BuildProject(services: IServiceCollection): void
    +InitializeModules(provider: IServiceProvider): void
}
```

```
class ProjectFactory {
    -logger: ILogger
    -modules: List<IModule>
    +BuildProject(services: IServiceCollection): void
    +InitializeModules(provider: IServiceProvider): void
    -OrderByDependencies(modules): List<IModule>
}
```

```
class LoggingModule {
    +ConfigureServices(services): void
    +Initialize(provider): void
}
```

```
class NotificationsModule {
    +ConfigureServices(services): void
    +Initialize(provider): void
}
```

```
class PaymentsModule {
    +ConfigureServices(services): void
    +Initialize(provider): void
}
```

```
class ModuleAttribute <<attribute>> {
    +Name: string
    +Dependencies: string[]
    +IsOptional: bool
}
```

IProjectFactory <|.. ProjectFactory

IModule <|.. LoggingModule

```
IModule <|.. NotificationsModule
IModule <|.. PaymentsModule
ProjectFactory o--> IModule : manages
ModuleAttribute ..> IModule : decorates
```

note right of ProjectFactory

Project Factory

- Descobre módulos dinamicamente
- Resolve ordem por dependências
- Orquestra registro e inicialização

end note

@enduml

DIAGRAMA 6: SEQUENCE - FACTORY METHOD

plantuml

@startuml FactoryMethodSequence

!theme plain

actor Client

participant "service:\nNotificationService" as Service

participant "notifier:\nINotifier" as Notifier

Client -> Service : NotifyCustomer(recipient, message)

activate Service

Service -> Service : CreateNotifier()

activate Service

note right: Factory Method\n(implementado por subclasse)

Service --> Service : notifier

deactivate Service

Service -> Notifier : Send(recipient, message)

activate Notifier

Notifier --> Service :

deactivate Notifier

Service --> Client :

deactivate Service

@enduml

DIAGRAMA 7: SEQUENCE - ABSTRACT FACTORY

plantuml

@startuml AbstractFactorySequence

!theme plain

actor Client

participant "generator:\nReportGenerator" as Generator

participant "factory:\nIReportFactory" as Factory

participant "header:\nIReportHeader" as Header

participant "table:\nIReportTable" as Table

participant "chart:\nIReportChart" as Chart

Client -> Generator : GenerateReport(title, data, chartData)

activate Generator

Generator -> Factory : CreateHeader()

activate Factory

Factory --> Generator : header

deactivate Factory

Generator -> Factory : CreateTable()

activate Factory

Factory --> Generator : table

deactivate Factory

Generator -> Factory : CreateChart()

activate Factory

Factory --> Generator : chart

deactivate Factory

Generator -> Header : Render(title)

activate Header

deactivate Header

Generator -> Table : Render(data)

activate Table

deactivate Table

Generator -> Chart : Render(chartData)

activate Chart

deactivate Chart

Generator --> Client :

deactivate Generator

note right of Factory

Todos os produtos vêm

da mesma factory,

garantindo consistência

end note

@enduml

DIAGRAMA 8: SEQUENCE - PROJECT FACTORY

plantuml

@startuml ProjectFactorySequence

!theme plain

actor "Application\nStartup" as App

participant "factory:\nProjectFactory" as Factory

participant "services:\nIServiceCollection" as Services

participant "module:\nIModule" as Module

App -> Factory : BuildProject(services)

activate Factory

Factory -> Factory : DiscoverModules()

activate Factory

Factory --> Factory : modules[]

deactivate Factory

Factory -> Factory : OrderByDependencies(modules)

activate Factory

Factory --> Factory : orderedModules[]

deactivate Factory

loop for each module

Factory -> Module : ConfigureServices(services)

activate Module

Module -> Services : AddTransient<T>()

Module -> Services : AddSingleton<T>()

Module --> Factory :

deactivate Module

end

Factory --> App :

deactivate Factory

App -> App : BuildServiceProvider()

App -> Factory : InitializeModules(provider)

activate Factory

loop for each loaded module

Factory -> Module : Initialize(provider)

activate Module

Module --> Factory :

deactivate Module

end

Factory --> App :

deactivate Factory

@enduml

DIAGRAMA 9: ARQUITETURA - SISTEMA COM FACTORIES (MERMAID)

mermaid

graph TB

subgraph "Presentation Layer"

API[API Controllers]

UI[UI Components]

end

subgraph "Application Layer"

Service[Application Services]

Factory[Factories]

end

subgraph "Domain Layer"

Entity[Domain Entities]

Interface[Domain Interfaces]

end

subgraph "Infrastructure Layer"

Impl[Concrete Implementations]

DI[DI Container]

end

API --> Service

UI --> Service

Service --> Factory

Factory --> DI

DI --> Impl

Impl -.implements.-> Interface

Service --> Interface

Factory -.creates.-> Interface

style Factory fill:#ffeb3b,stroke:#333,stroke-width:3px

style DI fill:#4caf50,stroke:#333,stroke-width:2px

DIAGRAMA 10: ARQUITETURA - FLUXO DE RESOLUÇÃO COM DI (MERMAID)

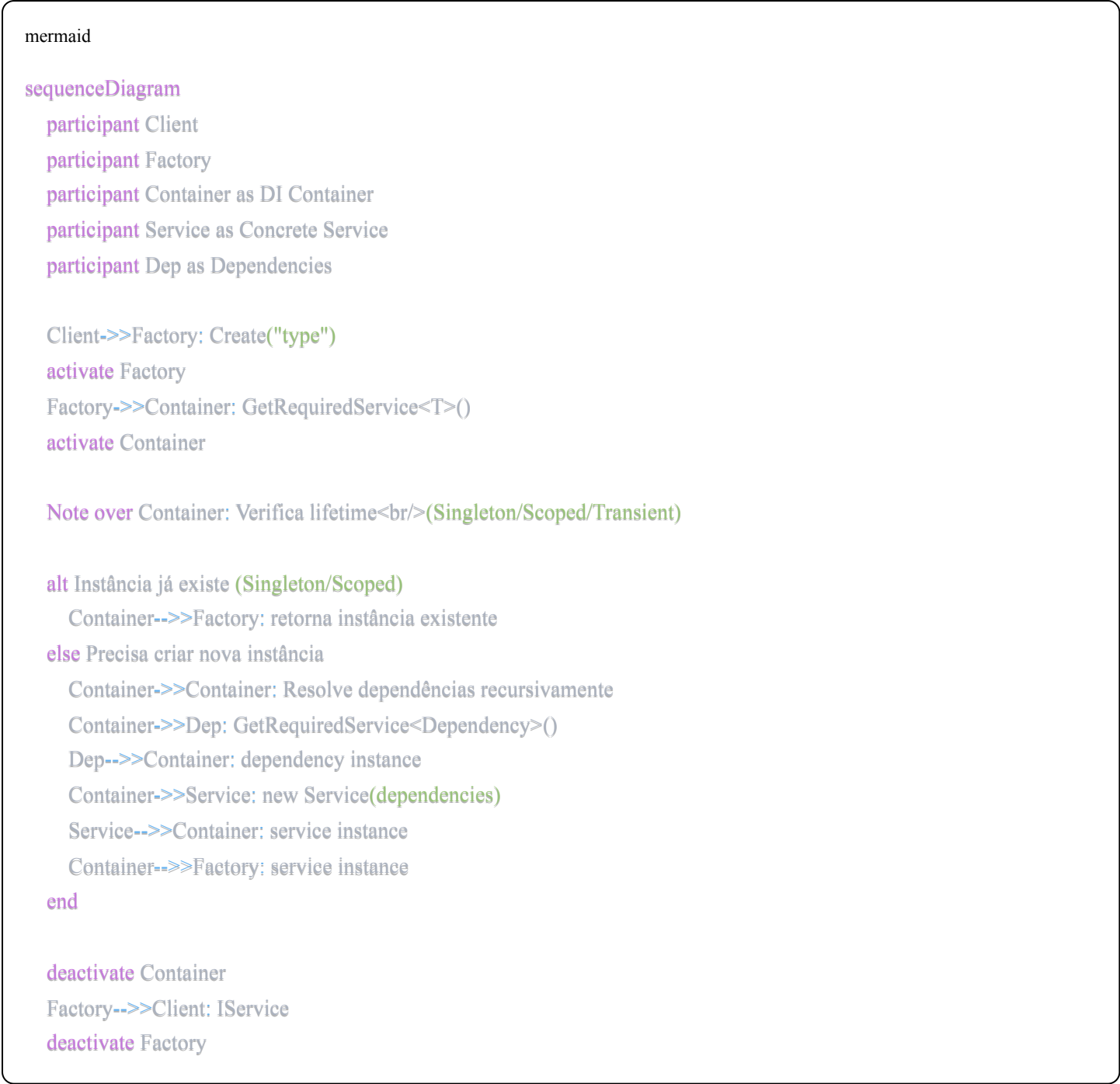


DIAGRAMA 11: COMPARAÇÃO DE PATTERNS (MERMAID)



```
graph LR
    subgraph "Simple Factory"
        SF[Static Method]
        SF --> P1[Product A]
        SF --> P2[Product B]
    end

    subgraph "Factory Method"
        FM[Abstract Creator]
        FM --> FC1[Concrete Creator 1]
        FM --> FC2[Concrete Creator 2]
        FC1 --> P3[Product A]
        FC2 --> P4[Product B]
    end

    subgraph "Abstract Factory"
        AF[Abstract Factory]
        AF --> CF1[Concrete Factory 1]
        AF --> CF2[Concrete Factory 2]
        CF1 --> P5[Product Family 1]
        CF2 --> P6[Product Family 2]
    end

    style SF fill:#ff9800
    style FM fill:#2196f3
    style AF fill:#9c27b0
```

DIAGRAMA 12: DECISÃO - QUANDO USAR QUAL PATTERN (MERMAID)

mermaid

graph TD

Start{Precisa criar
objetos?}

Start -->|Sim| Q1 {Tipos são
fixos e poucos?}

Q1 -->|Sim| Simple[Simple Factory
✓ Simples
✗ Não extensível]

Q1 -->|Não| Q2 {Subclasses precisam
customizar criação?}

Q2 -->|Sim| FactoryMethod[Factory Method
✓ Extensível via herança
✓ Template Method]

Q2 -->|Não| Q3 {Precisa criar
famílias de produtos?}

Q3 -->|Sim| AbstractFactory[Abstract Factory
✓ Consistência entre produtos
✓ Troca de família]

Q3 -->|Não| Q4 {Sistema é
modular/plugins?}

Q4 -->|Sim| ProjectFactory[Project Factory
✓ Descoberta dinâmica
✓ Gerenciamento de dependências]

Q4 -->|Não| Simple

style Simple fill:#ff9800

style FactoryMethod fill:#2196f3

style AbstractFactory fill:#9c27b0

style ProjectFactory fill:#4caf50

NOTAS DE USO

1. **Copie o código** de cada diagrama
2. **Cole no renderizador online** ou use extensão
3. **Exporte como PNG/SVG** para usar em apresentações
4. **Personalize cores/estilos** conforme necessário

Cores sugeridas para apresentações:

- Simple Factory: Laranja ( #ff9800)
- Factory Method: Azul ( #2196f3)
- Abstract Factory: Roxo ( #9c27b0)
- Project Factory: Verde ( #4caf50)
- DI Components: Verde escuro ( #4caf50)

